Vocabulary Building

simple_check

File: word.log

```
---
careful: ffttttt
assure: tttftt
accurate: fttfttt
curious: tfftttt
insurance: ttttt
carefree: fttttt
secure: ftfttftfft
sure: fttttft
ensure: tfftttt
care: fttfttft
curiosity: ttttt
pedestrian: ttttt
security: ttttft
insure: ttttt
pedicure: tttfttt
cure: tttttt
manicure: tttttt
cur: ttttftt
sur: ttttftt
accuracy: tttttt
```

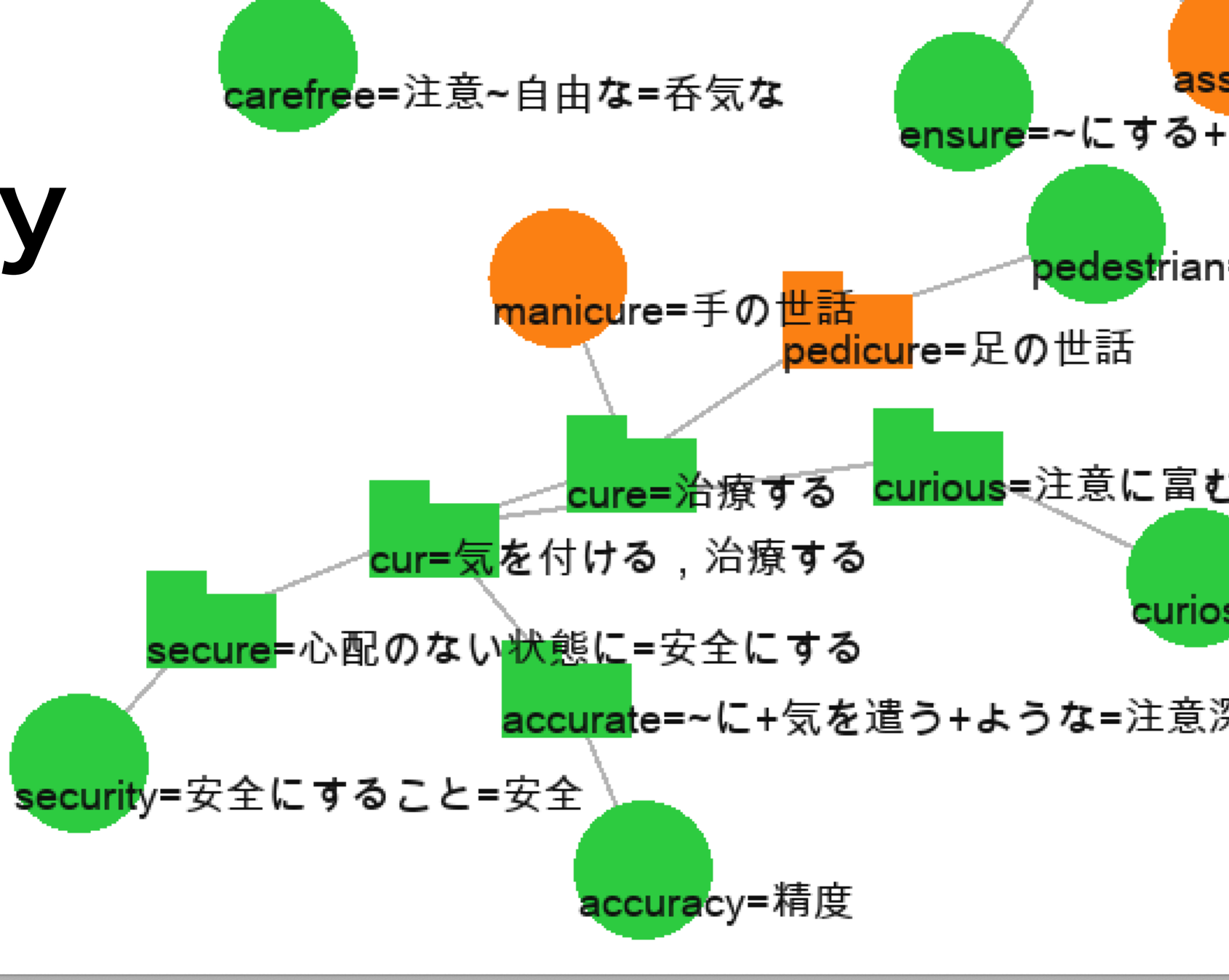
```
▶ pwd
/Users/bob/Desktop/lecture_25f/multi_scale_25f/ruby/voca_buil

▶ tree -d
.
└── _stack_init_word_log_251214
    ├── _stack_word_from_log_251214
    │   ├── _stack_behav_test_251214
    │   │   ├── _stack_method_class_251212
    │   │   │   └── _stack_log_251211
    │   │   │       └── _stack_check_251211
    │   │   │           └── _stack_chatGPT_251210
    │   │   ├── _stack_unit_test_251212
    │   │   └── test
    │   │       └── test_env
    │   └── test
    │       └── test_env
    └── test
        └── test_env

15 directories
```

```
▶ simple_check
ensure: ffftt
sure: tt
pedicure: tt
pedestrian: tt
accurate: tt
0: 足の世話, 2
1: 安全な, 確実な, 1
2: ~にする+安全, 0
3: 歩行者, 3
4: 精密な, 4
Q0: 'ensure' ? -> 2
Q1: 'sure' ? -> 1
Q2: 'pedicure' ? -> 0
Q3: 'pedestrian' ? -> 3
Q4: 'accurate' ? -> 4
2 true:  ["ensure", "~にする+安全"]
1 true:  ["sure", "安全な, 確実な"]
0 true:  ["pedicure", "足の世話"]
3 true:  ["pedestrian", "歩行者"]
4 true:  ["accurate", "~に+気を遣う+ような", "注意深く行なった", "精密な"]
```

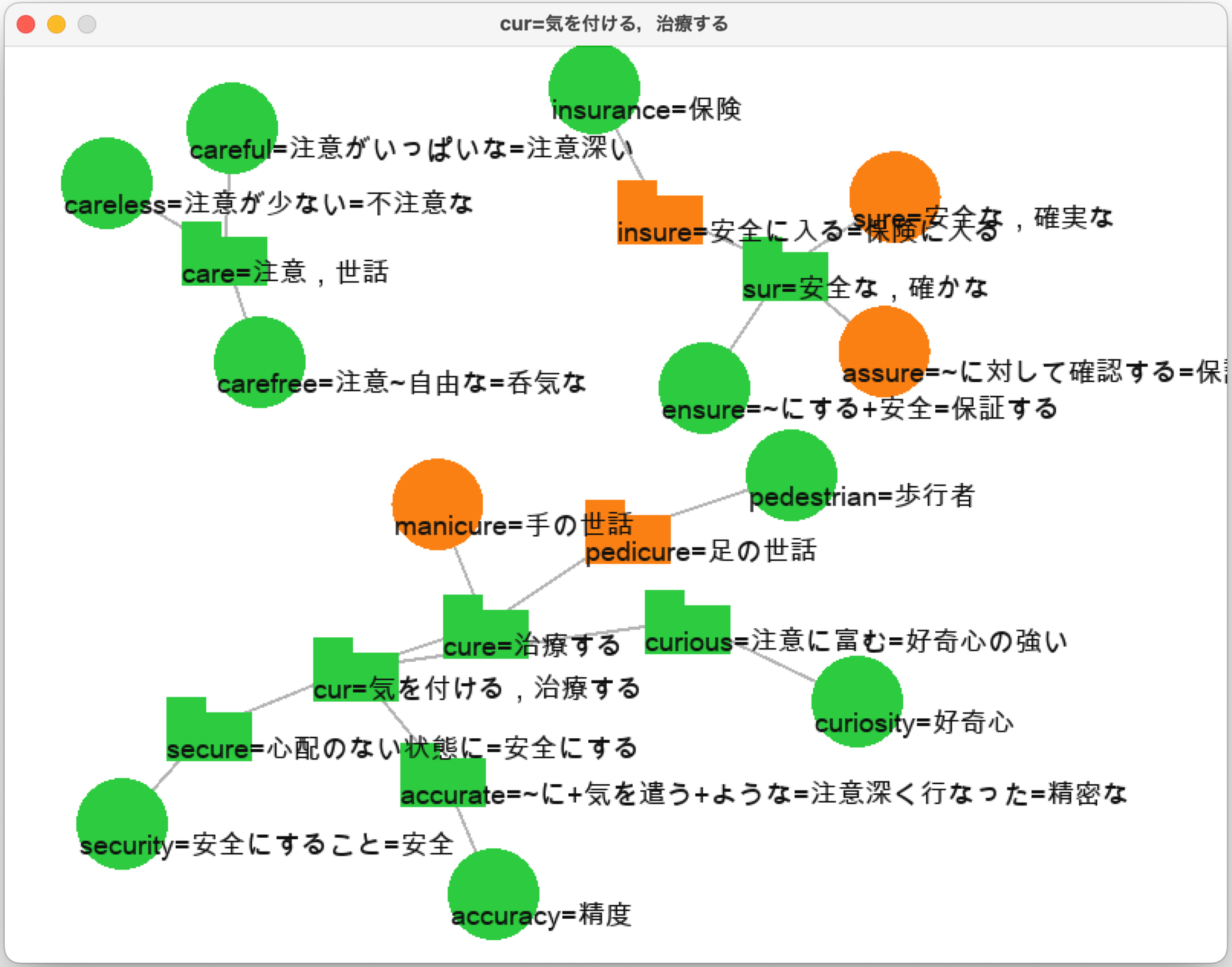
daddygongon 2025/12/14

Vocabulary Building

r21_cur



daddygongon 2025/12/


cur=気を付ける，治療する
insurance=保険
careful=注意がいっぱいな=注意深い
careless=注意が少ない=不注意な
insure=安全に入る=保険に入る
sure=安全な，確実な
care=注意，世話
sur=安全な，確かな
carefree=注意~自由な=呑気な
assure=~に対して確認する=保
ensure=~にする+安全=保証する
pedestrian=歩行者
manicure=手の世話
pedicure=足の世話
cure=治療する
curious=注意に富む=好奇心の強い
cur=気を付ける，治療する
curiosity=好奇心
secure=心配のない状態に=安全にする
accurate=~に+気を遣う+ような=注意深く行なった=精密な
security=安全にすること=安全
accuracy=精度

how to make cur_tree.yaml

```
▶ mk_sl -t cur_tree.yaml
mk_semi_lattice is running... with method mk_semi_lattice_viewer
{layer: 2, init_step: :from_tree, show_index: false, merge: false, file: "cur_tree.yaml", parent_dir: "/Users/bob/git_hub/hyper_cards/voca_buil", semi_dir: "/Users/bob/git_hub/hyper_cards/voca_buil/.semi_lattice", semi_lattice_yaml_path: "/Users/bob/git_hub/hyper_cards/voca_buil/.semi_lattice/semi_lattice.yaml"}
["ARGV", []]
["init_file", nil, :from_tree]
[".", "cur=気を付ける，治療する"]
[".//cur=気を付ける，治療する", :"secure=心配のない状態に=安全にする"]
[".//cur=気を付ける，治療する/secure=心配のない状態に=安全にする", :"security=安全にすること=安全"]
```

```
▶ bat cur_tree.yaml
File: cur_tree.yaml
---
cur=気を付ける，治療する:
  :secure=心配のない状態に=安全にする:
    :security=安全にすること=安全:
  :cure=治療する:
    :manicure=手の世話:
    :pedicure=足の世話:
      :pedestrian=歩行者:
  :accurate=~に+気を遣う+ような=注意深く行なった=精密な:
    :accuracy=精度:
  :curious=注意に富む=好奇心の強い:
    :curiosity=好奇心:
care=注意，世話:
  :careful=注意がいっぱいな=注意深い:
  :careless=注意が少ない=不注意な:
  :carefree=注意~自由な=呑気な:
sur=安全な，確かな:
  :sure=安全な，確実な:
  :ensure=~にする+安全=保証する:
  :insure=安全に入る=保険に入る:
    :insurance=保険:
  :assure=~に対して確認する=保証する:
```

#1 chatGPT_251210

```
▶ ruby mk_test.rb cur_tree.yaml
e_words:
1: pedicure
2: cure
3: insurance
4: assure
5: manicure
6: care
7: cur
8: carefree
9: accuracy
10: secure
11: curious
12: careful
13: accurate
14: sur
15: pedestrian
16: insure
17: sure
18: ensure
```

```
j_words:
a: 注意，世話
b: 呑気な
c: 安全
d: 好奇心の強い
e: 精度
f: 安全な，確かな
g: 保険に入る
h: 不注意な
i: 手の世話
j: 足の世話
k: 保証する
l: 歩行者
m: 治療する
n: 保険
o: 保証する
p: 安全な，確実な
q: 好奇心
```

```
ペア問い合わせテスト:
Q1: 'pedicure' の意味は？ → '注意,
Q2: 'cure' の意味は？ → '呑気な'
Q3: 'insurance' の意味は？ → '安全
Q4: 'assure' の意味は？ → '好奇心の
Q5: 'manicure' の意味は？ → '精度
Q6: 'care' の意味は？ → '安全な，確
Q7: 'cur' の意味は？ → '保険に入る'
Q8: 'carefree' の意味は？ → '不注
Q9: 'accuracy' の意味は？ → '手の
Q10: 'secure' の意味は？ → '足の世
Q11: 'curious' の意味は？ → '保証
Q12: 'careful' の意味は？ → '歩行
Q13: 'accurate' の意味は？ → '治療
Q14: 'sur' の意味は？ → '保険'
Q15: 'pedestrian' の意味は？ → '
```

wrong code by ChatGPT

```
▶ bat mk_test.rb -p
file = ARGV[0] || 'dir_tree.yaml'
lines = File.readlines(file)
# require 'yaml'
# YAML.load(File.read(file)).flatten(-1)は絶望的に失敗

e_words = []
j_words = []

lines.each do |line|
  # 先頭の空白（スペースやタブ）を許容
  if line =~ /^\s*:?([a-zA-Z]+)=(.+?):?$/
    key = $1
    value = $2
    parts = "#{key}=#{value}".split('=')
    e_words << parts.first
    j_words << parts.last
  end
end
```

```
# ランダムにシャッフル
e_words_shuffled = e_words.shuffle
j_words_shuffled = j_words.shuffle

# 頭に番号やアルファベットを付与して出力
puts "e_words:"
e_words_shuffled.each_with_index do |word, idx|
  puts "#{idx + 1}: #{word}"
end

puts "\nj_words:"
j_words_shuffled.each_with_index do |word, idx|
  puts "#{('a'.ord + idx).chr}: #{word}"
end

# ペア問い合わせテスト
puts "\nペア問い合わせテスト:"
e_words_shuffled.each_with_index do |ew, idx|
  jw = j_words_shuffled[idx]
  puts "Q#{idx + 1}: '#{ew}' の意味は？ → '#{jw}'"
end
```

read dir_tree.yaml and puts

```
bat test.rb -p
file = ARGV[0] || 'cur_tree.yaml'
lines = File.readlines(file)

words = []
lines[1..5].each do |line|
  line_strip = if line.strip[0]==':'
                 line.strip[1..-2]
               else
                 line.strip[0..-2]
               end
  tmp = line_strip.split("=")
  words << tmp
end
```

```
▶ ruby test.rb cur_tree.yaml
0: 手の世話, 2
1: 治療する, 1
2: 気を付ける, 治療する, 4
3: 安全にする, 3
4: 安全, 0
Q0: 'security' ? -> Q1: 'cure' ? -> Q2: 'manicure' ? -> Q3: 'secure' ? ->
Q4: 'cur' ? -> [["security", "安全にすること", "安全"], "---", false]
[["cure", "治療する"], "cur=気を付ける, 治療する:", false]
[["manicure", "手の世話"], "  :secure=心配のない状態に=安全にする:", true]
[["secure", "心配のない状態に", "安全にする"], "    :security=安全にすること=安全:",
false]
[["cur", "気を付ける, 治療する"], "  :cure=治療する:", false]
```

#2 check_251211

```
ruby simple_check.rb  cur_dir.yaml
ruby simple_check.rb
```

#<= FILEがgets.chompの異常動作の元

```
0: 注意, 世話, 2
1: 精密な, 3
2: 好奇心の強い, 0
3: 保険に入る, 4
4: 足の世話, 1
curious ? -> 2
pedicure ? -> 4
care ? -> 0
accurate ? -> 1
insure ? -> 3
2 true:  ["curious", "注意に富む", "好奇心の強い"]
4 true:  ["pedicure", "足の世話"]
0 true:  ["care", "注意, 世話"]
1 true:  ["accurate", "~に+気を遣う+ような", "注意深く行なった", "精密な"]
3 true:  ["insure", "安全に入る", "保険に入る"]
```

```ruby
user_inputs = []
words_shuffled.each_with_index do |ew, idx|
  print "#{ew[0]} ? -> "
  user_inputs << gets.chomp
end

require 'colorize'
results = {}
t_or_f = nil
words_shuffled.each_with_index do |ans, i|
  user_num = user_inputs[i].to_i
  print "%-2s" % user_num
  if user_num <= answers_shuffled.size
    ans_num = answers_shuffled[user_num][-1]
  else
    ans_num = -1
  end
  if ans == words_shuffled[ans_num]
    print "true:  ".green
    t_or_f = 't'
  else
    print "false: ".red
    t_or_f = 'f'
  end
  p ans
  results[ans[0]] = t_or_f
end
```

usage

input

output

```ruby
w_num = ARGV[1] || 4
a_num = ARGV[2] || 4
words_shuffled = words.shuffle[0..w_num]
#pp words_shuffled[0..5]

answers = []
words_shuffled.each_with_index do |word, idx|
  answers << [word[-1], idx]
end

answers_shuffled=answers.shuffle[0..a_num]
answers_shuffled.each_with_index do |word, idx|
  puts "#{idx}: #{word[0]}, #{word[-1]}"
end

user_inputs = []
words_shuffled.each_with_index do
  print "Q#{idx }: '#{ew[0]}' ? ->
  user_inputs << gets.chomp
end
```

```
▶ ruby simple_check.rb
0: 呑気な, 2
1: 保険に入る, 1
2: 気を付ける, 治療する, 4
3: 精密な, 3
4: 手の世話, 0
Q0: 'manicure' ? -> 4
Q1: 'insure' ? -> 1
Q2: 'carefree' ? -> 0
Q3: 'accurate' ? -> 3
Q4: 'cur' ? -> 2
```

```ruby
require 'colorize'
results = {}
t_or_f = nil
words_shuffled.each_with_index do |ans, i|
  user_num = user_inputs[i].to_i
  print "%-2s" % user_num
  if user_num <= answers_shuffled.size
    ans_num = answers_shuffled[user_num][-1]
  else
    ans_num = -1
  end
  if ans == words_shuffled[ans_num]
    print "true:  ".green
    t_or_f = 't'
  else
    print "false: ".red
    t_or_f = 'f'
  end
  p ans
  results[ans[0]] = t_or_f
end
```

```
4 true:  ["manicure", "手の世話"]
1 true:  ["insure", "安全に入る", "保険に入る"]
0 true:  ["carefree", "注意~自由な", "呑気な"]
3 true:  ["accurate", "~に+気を遣う+ような", "注意深く行なった"
2 true:  ["cur", "気を付ける, 治療する"]
```

#3 log_251211

```
▶ cat word.log
---
careful: ff
assure: tt
accurate: ft
curious: tf
insurance: tt
carefree: ftttt
secure: ft
sure: f
ensure: t
care: f
curiosity: tt
pedestrian: tt
security: tttt
insure: ttt
pedicure: t
cure: t
manicure: t
cur: t
```

word.log, check.logを作成

```ruby
require 'YAML'

check_log_file = './check.log'
word_log_file = './word.log'
check_log = File.exist?(check_log_file) ?
              YAML.load_file(check_log_file,
                             permitted_classes: [Date, Time]) :
              []
word_log = File.exist?(check_log_file) ?
             YAML.load(File.read(word_log_file)) :
             results

check_log << Time.now
# p check_log
results.each do |key, val|
  if word_log[key] == nil
    word_log[key] = val
  else
    word_log[key] += val
  end
end

File.write(check_log_file, YAML.dump(check_log))
File.write(word_log_file, YAML.dump(word_log))
```

#4 method_class_251212

method,
- それぞれの作業内容をmethodに分けてmainで実行するように変更.

optparse,
- optparseでoptionを操作するように変更
- shebangで起動するように修正.

class,
- 全てのmethodをclass SimpleVocCheckとしてnew.runで動くように修正

a+x

```ruby
#!/usr/bin/env ruby
require 'optparse'
require 'colorize'
require 'YAML'

class SimpleVocCheck
  def initialize
    @options = {…
    }
    @check_log_file = './check.log'
    @word_log_file = './word.log'
  end

  def parse_options…
  end

  def read_words(file)…
  end

  def shuffle_words(words, w_num)…
  end

  def make_answers(words_shuffled)…
  end

  def shuffle_answers(answers, a_num)…
  end

  def print_answers(answers_shuffled)…
  end

  def get_user_inputs(words_shuffled)…
  end

  def check_results(words_shuffled, answers_shuffled, user_inputs)…
  end

  def update_logs(results)…
  end

  def run
    parse_options
    words = read_words(@options[:file])
    words_shuffled = shuffle_words(words, @options[:w_num])
    answers = make_answers(words_shuffled)
    answers_shuffled = shuffle_answers(answers, @options[:a_num])
    print_answers(answers_shuffled)
    user_inputs = get_user_inputs(words_shuffled)
    results = check_results(words_shuffled, answers_shuffled, user_inputs)
    update_logs(results)
  end
end

if __FILE__ == $0
  SimpleVocCheck.new.run
end
```

#5 behav_test_251214

```
▶ tree -L 2
.
├── check.log
├── cur_tree.yaml
├── readme.org
├── simple_check
├── test
│   ├── simple_check_test.rb
│   └── test_env
├── voca_buil.key
├── voca_buil.pdf
└── word.log

6 directories, 23 files
```

```
▶ ruby test/simple_check_test.
Started with run options --see

SimpleVocCheckTest
  test_it_does_something_usefu
PASS (0.00s)
  test_hello_command_outputs_h
PASS (0.03s)

Finished in 0.02752s
2 tests, 4 assertions, 0 failu
```

* bddの原案作成
振る舞いテストという言葉悪いのか，思った通りのtestができないので，
git_hub/hyper_cards/mk_hyper_card/test/test_mk_hyper_card.rbを
参照して作成.

```
▶ bat test/simple_check_test.rb -p
require 'minitest/autorun'
require 'minitest/reporters'
require 'fileutils'
require 'yaml'
require 'open3'
#require_relative '../simple_check'

Minitest::Reporters.use! Minitest::Reporters::SpecReporter.new

class SimpleVocCheckTest < Minitest::Test
  TEST_DIR = File.expand_path('test_env', __dir__)
  TEST_YAML = File.join(TEST_DIR, 'cur_tree.yaml')
  CHECK_LOG = File.join(TEST_DIR, 'check.log')
  WORD_LOG = File.join(TEST_DIR, 'word.log')

  def setup
    FileUtils.rm_rf(TEST_DIR) if Dir.exist?(TEST_DIR)
    FileUtils.mkdir_p(TEST_DIR, verbose: true)
  end

  def teardown
#    FileUtils.rm_rf(TEST_DIR, verbose: true) if Dir.exist?(TEST_DIR)
  end

  def test_it_does_something_useful
    assert true
  end

  def test_hello_command_outputs_hello
    Dir.chdir(TEST_DIR) do
      p ["#1 TEST_DIR", Dir.pwd]
      #      p comm = "ls ../../"
      p comm = "../../simple_check -h"
      stdout, stderr, status = Open3.capture3(comm)
      assert status.success?, "'#{comm}' failed: #{stderr}"
      assert_match(/^Usage:/, stdout)
    end
  end
end
```

```
▶ tree -d
.
├── _stack_method_class_2512
│   └── _stack_log_251211
│       └── _stack_check_251
│           └── _stack_chatG
├── _stack_unit_test_251212
└── test
    └── test_env

8 directories
```

#6 word_from_log_251214

▶ simple_check

```
secure: ftfttftff
cur: ttttft
manicure: ttttt
cure: ttttt
pedicure: tttftt
0: 気を付ける，治療する， 1
1: 手の世話， 2
2: 安全にする， 0
3: 足の世話， 4
4: 治療する， 3
Q0: 'secure' ? -> 2
Q1: 'cur' ? -> 0
Q2: 'manicure' ? -> 1
Q3: 'cure' ? -> 4
Q4: 'pedicure' ? -> 3
2 true:  ["secure", "心配のない状態に", "安全にする"]
0 true:  ["cur", "気を付ける，治療する"]
1 true:  ["manicure", "手の世話"]
```

```ruby
def run
  parse_options
  words_shuffled = if File.exist?(@word_log_file)
                     words = read_words(@options[:file])
                     select_least_correct_words(words)
                   else
                     words = read_words(@options[:file])
                     shuffle_words(words, @options[:w_num])
                   end
  answers = make_answers(words_shuffled)
```

```ruby
def run
  parse_options

  words = read_words(@options[:file])

  words_shuffled = shuffle_words(words, @options[:w_num]

  answers = make_answers(words_shuffled)
```

```ruby
def select_least_correct_words(whole_words)
  # word.yamlの全てのwordがword.logにある場合はこれでOK
  # check初期のword.log<word.yamlではより適切なword抽出が必要
  # word.logがない場合はnilで作ってほうが楽かも.
#  pp whole_words
  return unless File.exist?(@word_log_file)
  word_log = YAML.load_file(@word_log_file)
  # 't'の数が少ない順に5個
  least5 = word_log.sort_by { |k, v| v.count('t') }.first(5)
  selected_words=[]
  least5.each do |sel_word, result|
    puts "#{sel_word}: #{result}"
    whole_words.each do |word|
      if word[0] == sel_word
        selected_words << word
      end
    end
  end
#  pp selected_words
  selected_words
end
```

#7 init_word_log_251214

- 'word.log'が存在しない時に,
  - wordsからHash word_logを作成し,
  - そのkeyをword[0]に, valをnilに設定.
  - それを'word_log'にYAML形式で保存するように修正
- vがnilの時にも対応するように修正

```
▶ simple_check
assure: f
ensure: f
sur:
carefree:
careless:
0: 保証する, 1
1: 保証する, 0
2: 安全な, 確かな, 2
3: 呑気な, 3
4: 不注意な, 4
Q0: 'assure' ? -> 0
Q1: 'ensure' ? -> 1
Q2: 'sur' ? -> 2
Q3: 'carefree' ? -> 3
Q4: 'careless' ? -> 4
0 false: ["assure", "~に
1 false: ["ensure", "~に
2 true: ["sur", "安全な, 確かな"]
```

```
127: class SimpleVocCheck |

127|   return unless File.exist?(@word_log_file)
128|   word_log = YAML.load_file(@word_log_file)
129|   # 't'の数が少ない順に5個
130|   least5 = word_log.sort_by { |k, v| v&.count('t') || 0 }.first(5)
131|   selected_words=[]
132|   least5.each do |sel_word, result|
133|     puts "#{sel_word}: #{result}"
```

```
127|   return unless File.exist?(@word_log_file)
128|   word_log = YAML.load_file(@word_log_file)
129|   # 't'の数が少ない順に5個
130|   least5 = word_log.sort_by { |k, v| v.count('t') }.first(5
131|   selected_words=[]
132|   least5.each do |sel_word, result|
133|     puts "#{sel_word}: #{result}"
```

```
141: class SimpleVocCheck |

141|   selected_words
142| end
143|
144| def init_word_log(words)
145|   word_log = {}
146|   words.each do |word|
147|     word_log[word[0]] = nil
148|   end
149|   word_log
150| end
151|
152| def run
153|   parse_options
154|   words_shuffled = if File.exist?(@word_log_file)
```

```
141|   selected_words
142| end
143|
144| def run
145|   parse_options
146|   words_shuffled = if File.exist?(@word_log_file)
```

```
148: class SimpleVocCheck |

156|     select_least_correct_words(words)
157|   else
158|     words = read_words(@options[:file])
159|     mk_log_file = File.write(@word_log_file, YAML.dump(init_word_log(words)))
160|     shuffle_words(words, @options[:w_num])
```

```
148|     select_least_correct_words(words)
149|   else
150|     words = read_words(@options[:file])
151|     shuffle_words(words, @options[:w_num])
```